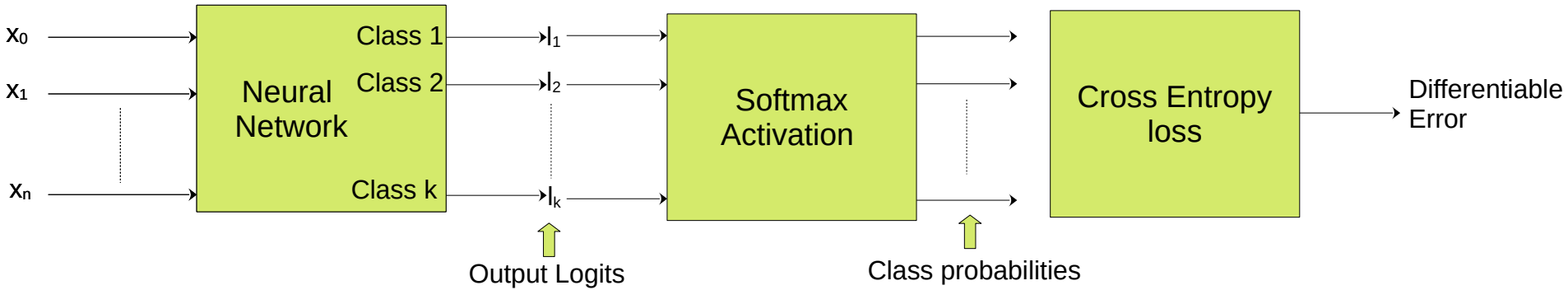
Softmax and Cross Entropy in Classification

Softmax and Cross Entropy in Classification

Goals:

- Explain what softmax does and why we use it
- Define cross-entropy and its gradient
- Implement the pair correctly for multi-class classification
- Avoid common pitfalls and handle class imbalance/calibration

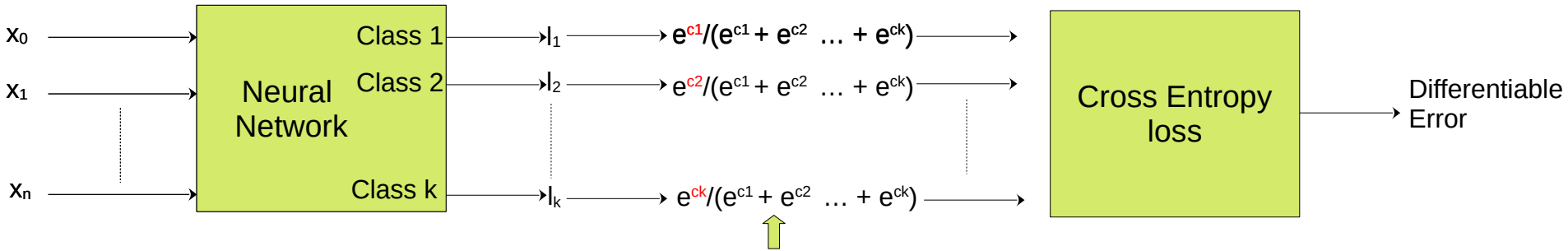
Where They Fit in a Classifier



- **Logits**: arbitrary real scores from model (unnormalized)
- Largest logit is the winner
- **Softmax** outputs class ‘probabilities’
- **Cross Entropy** how wrong we are.

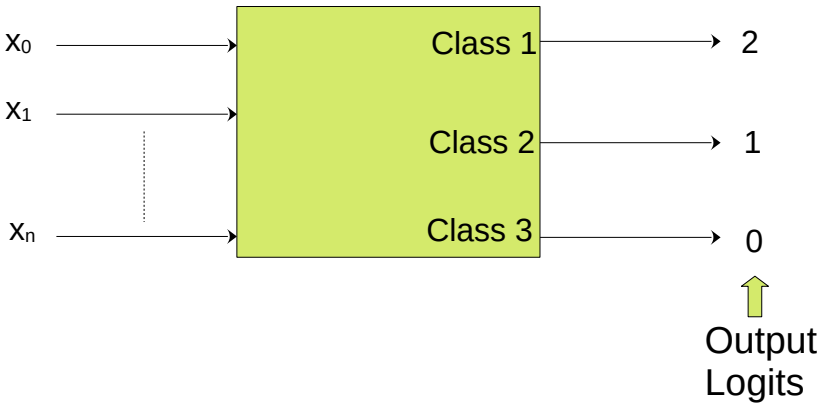
Softmax Activation

$$\frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

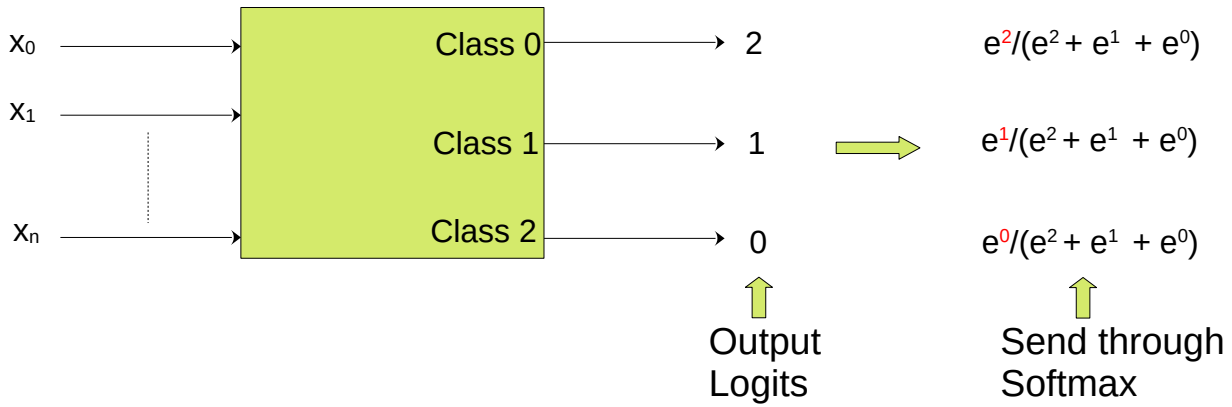


- Turns raw scores (“logits”) into a ‘probability’ like distribution (sum to 1)
- Differentiable
- Largest softmaxed logit is still largest
- In fact softmax pushes largest higher and others lower

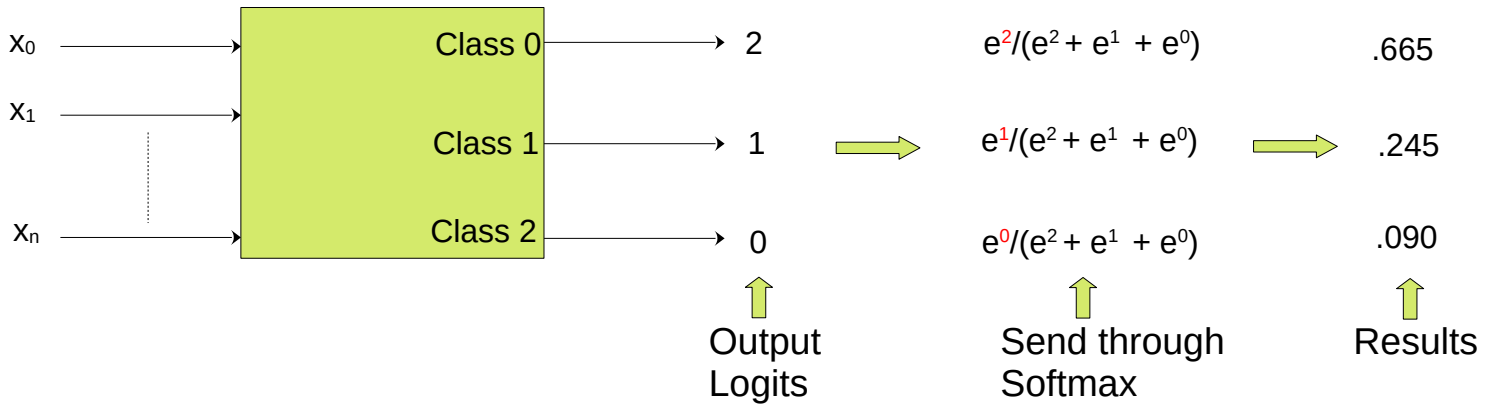
Softmax Activation Example



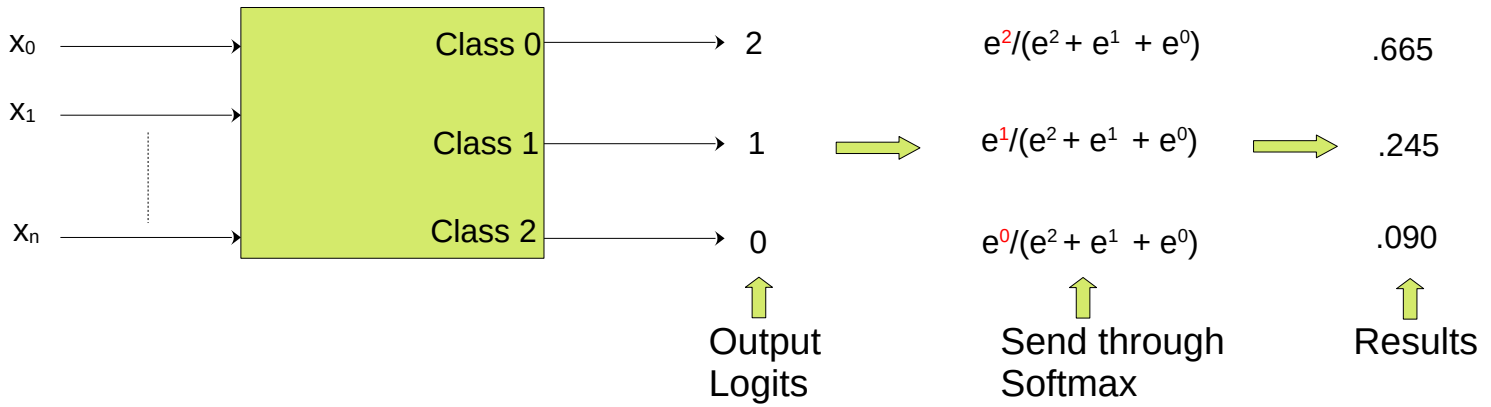
Softmax Activation Example



Softmax Activation Example

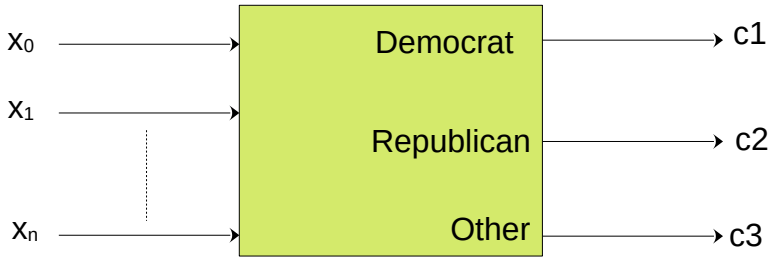


Softmax Activation Example



- .665 + .245 + .090=1
- Class 0 is still selected
- Class 0 probability is much larger than the original gap between Class0 and Class 1 suggests

Softmax Classification Example
use argmax to select predicted class

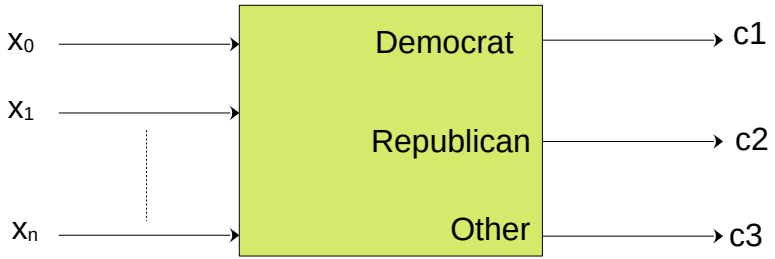


A NN with a final layer softmax activation function.
Three training rows are fed in with the following results

c1	c2	c3	targets	correct?
0.3	0.3	0.4	0 0 1 (democrat)	yes
0.3	0.4	0.3	0 1 0 (republican)	yes
0.1	0.2	0.7	1 0 0 (other)	no

It has a classification accuracy of 2/3 or 66%
It has a classification error of 1/3 or 33%
It barely gets the first 2 right and is way off on the third.

Softmax Classification Example
use argmax to select predicted class



A NN with a final layer softmax activation function.
Three training rows are fed in with the following results

c1	c2	c3	targets	correct?
0.3	0.3	0.4	0 0 1 (democrat)	yes
0.3	0.4	0.3	0 1 0 (republican)	yes
0.1	0.2	0.7	1 0 0 (other)	no

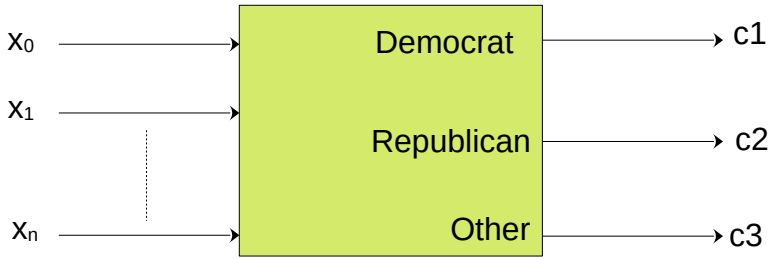
It has a classification accuracy of 2/3 or 66%
It has a classification error of 1/3 or 33%
It barely gets the first 2 right and is way off on the third.

A different NN with a final layer softmax activation function.
Three training rows are fed in with the following results

c1	c2	c3	targets	correct?
0.1	0.2	0.7	0 0 1 (democrat)	yes
0.1	0.7	0.2	0 1 0 (republican)	yes
0.3	0.4	0.3	1 0 0 (other)	no

It has a classification accuracy of 2/3 or 66%
It has a classification error of 1/3 or 33%
It nails the first 2 and barely misses the third.

Softmax Classification Example
use argmax to select predicted class



A NN with a final layer softmax activation function.
Three training rows are fed in with the following results

c1	c2	c3	targets	correct?
0.3	0.3	0.4	0 0 1 (democrat)	yes
0.3	0.4	0.3	0 1 0 (republican)	yes
0.1	0.2	0.7	1 0 0 (other)	no

It has a classification accuracy of 2/3 or 66%
It has a classification error of 1/3 or 33%
It barely gets the first 2 right and is way off on the third.

A different NN with a final layer softmax activation function.
Three training rows are fed in with the following results

c1	c2	c3	targets	correct?
0.1	0.2	0.7	0 0 1 (democrat)	yes
0.1	0.7	0.2	0 1 0 (republican)	yes
0.3	0.4	0.3	1 0 0 (other)	no

It has a classification accuracy of 2/3 or 66%
It has a classification error of 1/3 or 33%
It nails the first 2 and barely misses the third.

The second network is clearly better but the metrics do not reflect that
Also, argmax is not differentiable so cannot backprop through it

Cross Entropy Classification Error

- Also called the negative log likelyhood
- Takes the negative log of the softmax score for ONLY the correct class. All other entries are ignored.
- So applying cross entropy error for the previous 2 examples;

c1	c2	c3	targets	correct?
0.3	0.3	0.4	0 0 1 (democrat)	yes
0.3	0.4	0.3	0 1 0 (republican)	yes
0.1	0.2	0.7	1 0 0 (other)	no

Row1 error=-((ln(0.3)*0) + (ln(0.3)*0) + (ln(0.4)*1)) = -ln(0.4)
Row2 error=-ln(.4)
Row3 error=-ln(.1)

Total error=-(ln(0.4) + ln(0.4) + ln(0.1)) / 3 = 1.38

c1	c2	c3	targets	correct?
0.1	0.2	0.7	0 0 1 (democrat)	yes
0.1	0.7	0.2	0 1 0 (republican)	yes
0.3	0.4	0.3	1 0 0 (other)	no

Row1 error=-((ln(0.1)*0) + (ln(0.2)*0) + (ln(0.7)*1)) = -ln(0.7)
Row2 error=-ln(.7)
Row3 error=-ln(.3)

Total error=-(ln(0.7) + ln(0.7) + ln(0.3)) / 3 = 0.64

Cross Entropy Classification Error
Why it is better

- Also called the negative log likelyhood
- Takes the negative log of the softmax score for ONLY the correct class. All other entries are ignored.
- So applying cross entropy error for the previous 2 examples;

c1	c2	c3	targets	correct?
0.3	0.3	0.4	0 0 1 (democrat)	yes
0.3	0.4	0.3	0 1 0 (republican)	yes
0.1	0.2	0.7	1 0 0 (other)	no

Row1 error=-((ln(0.3)*0) + (ln(0.3)*0) + (ln(0.4)*1)) = -ln(0.4)
Row2 error=-ln(.4)
Row3 error=-ln(.1)

Total error=-(ln(0.4) + ln(0.4) + ln(0.1)) / 3 = 1.38

c1	c2	c3	targets	correct?
0.1	0.2	0.7	0 0 1 (democrat)	yes
0.1	0.7	0.2	0 1 0 (republican)	yes
0.3	0.4	0.3	1 0 0 (other)	no

Row1 error=-((ln(0.1)*0) + (ln(0.2)*0) + (ln(0.7)*1)) = -ln(0.7)
Row2 error=-ln(.7)
Row3 error=-ln(.3)

Total error=-(ln(0.7) + ln(0.7) + ln(0.3)) / 3 = 0.64

The error now reflects that the second network is much better than the first

Pytorch Cross Entropy Example

```
criterion = nn.CrossEntropyLoss()
```

```
def train(model, train_loader, test_loader, optimizer, epochs=10)
    for ep in range(1, epochs+1):
        model.train()
        run_loss, n = 0.0, 0
        for xb, yb in train_loader:
            xb, yb = xb.to(device), yb.to(device)
            optimizer.zero_grad(set_to_none=True)
            logits = model(xb)
            loss = criterion(logits, yb)
            loss.backward()
```

Note: pytorch cross entropy will automatically
apply softmax before it calculates the cross entropy

When not to use Cross Entropy

- Multi-label problems (classes are not mutually exclusive)
- Instead feed logits into BCEWithLogitsLoss

Summary

- For single label classification use cross entropy loss
- BTW it automatically applys softmax to the output logits
- For multilabel classification use BCEWithLogitsLoss